

VirtuEx 성능 개선 감사 보고서 (6차)

VirtuEx Performance Improvement Audit Report (6th)

- 감사일: 2026-03-16
- 감사 범위: 프론트엔드 혹은 memoization, API 호출 패턴, WebSocket 메시지 처리, 번들 최적화, 이미지/폰트 로딩, SSR/hydration, DB 쿼리 패턴 (전 서비스)
- 감사 방법: 소스 코드 정적 분석, 렌더링 패턴 추적, 쿼리 패턴 추적, 번들/캐싱 전략 검토
- 심각도 기준: Critical(즉시 수정) / High(1주 내) / Medium(2주 내) / Low(개선 권장)
- 비고: 발견 사항 정리만 수행하며, 수정은 포함하지 않음

이전 감사 대비 수정 현황 (5차 → 6차)

이전 이슈	상태	비고
[OE-H-01] 오더북 삽입 시 전체 정렬 O(N log N)	수정 됨	이진 탐색(binarySearchInsert) + splice로 O(N) 삽입으로 변경
[OE-M-01] toRemove.includes() O(N*M)	수정 됨	Set 기반 O(N) 필터링으로 변경
[FC-M-01] 포트폴리오 혹은 staleTime 미설정	수정 됨	staleTime: 8000 추가
[FC-M-02] 주문 내역 혹은 폴링+WS 중복	수정 됨	staleTime: 8000 추가
[WS-L-01] 재접속 지수 백오프 미적용	수정 됨	reconnectionDelayMax: 10000, randomizationFactor: 0.5 적용
[FR-H-01] AssetList useMemo 의존성 과다	미수정	추후 개선 예정
[FR-H-02] 리더보드 FLIP 애니메이션 리플로우	미수정	추후 개선 예정

요약 (Executive Summary)

카테고리	Critical	High	Medium	Low	합계
A. 프론트엔드 렌더링/Memoization	0	2	3	1	6
B. API 호출 패턴	0	0	2	1	3
C. WebSocket 메시지 처리	0	0	1	1	2
D. 번들/코드 분할	0	0	1	1	2
E. DB 쿼리 패턴	0	1	1	0	2
F. 이미지/폰트 로딩	0	0	1	1	2
G. SSR/Hydration	0	0	0	1	1

합계	0	3	9	6	18
----	---	---	---	---	----

A. 프론트엔드 렌더링/Memoization — 6건

[FR-H-01] AssetList — useMemo 내부 정렬이 every tick마다 O(N log N) (High)

현상: AssetList.tsx 의 filtered useMemo에서 assets 배열이 변경될 때마다(WebSocket 가격 업데이트 포함) 전체 정렬이 실행됨. 쓰로틀(3초)이 적용되어 있으나, assets 참조 자체가 변경되면 useMemo가 재실행되어 쓰로틀이 무효화 되는 경우 발생 **위치:** frontend/src/components/market/AssetList.tsx:92-150 **영향:** 100개+ 종목에서 WebSocket 업데이트마다 불필요한 정렬+필터 연산 **권장:** assets 의 참조 안정성 확보 (상위 컴포넌트에서 useMemo로 래핑) 또는 sortedOrderRef 갱신 조건을 실제 데이터 변경 여부로 판별 **심각도:** High

[FR-H-02] 리더보드 FLIP 애니메이션 — 모든 행에 대해 개별 DOM 조작 (High)

현상: LeaderboardPage.tsx:245-280 에서 순위 변동 시 각 행에 개별적으로 el.style 조작 수행. void movedEls[0].offsetHeight 단일 리플로우는 적용되었으나, sortedLeaderboard 변경마다 전체 행(최대 100+)을 순회하며 ref 조회+비교 수행 **위치:** frontend/src/app/(main)/leaderboard/page.tsx:245-280 **영향:** 대규모 리더보드에서 매 폴링(10초)마다 DOM 스타일 조작 비용 **권장:** CSS will-change: transform 미리 설정, 또는 변경된 행만 선별적 FLIP 적용 **심각도:** High

[PF-M-01] 커뮤니티 페이지 — 단일 컴포넌트에 과도한 상태 (Medium)

현상: CommunityPage 컴포넌트가 950줄 이상으로, discussions/strategies/traders/feed 4개 탭의 상태를 모두 관리. 탭 전환 시 비활성 탭의 상태 변경도 리렌더링 유발. useLeaderboard, useFollowing, useCopyTradeStatus 등이 모든 탭에서 항상 실행 **위치:** frontend/src/app/(main)/community/page.tsx:296-954 **영향:** 자유게시판 탭에서도 리더보드/팔로우/카피트레이딩 데이터 페칭 발생 **권장:** 각 탭을 독립 컴포넌트로 분리, enabled 조건으로 비활성 탭 데이터 페칭 방지 **심각도:** Medium

[PF-M-02] OrderForm — useEffect 내 상태 연쇄 업데이트 (Medium)

현상: OrderForm.tsx 에서 currencyMode / rate 변경 시 useEffect 로 price 와 triggerPrice 를 각각 업데이트하는 두 개의 독립 Effect가 있음. 각 setState 호출이 별도 렌더를 트리거하여 통화 전환 시 2~3회 연속 리렌더링 **위치:** frontend/src/components/trading/OrderForm.tsx:104-129 **영향:** 통화 모드 전환 시 불필요한 중간 렌더링 **권장:** 두 Effect를 하나로 통합하거나, useReducer 로 일괄 상태 업데이트 **심각도:** Medium

[PF-M-03] DashboardPage — displayAssets 재계산 체인 (Medium)

현상: assets → displayAssets → filteredDisplayAssets 3단계 useMemo 체인에서, livePrices 변경 시 중간 displayAssets 가 새 배열을 생성하여 하위 filteredDisplayAssets 도 연쇄 재계산. displayAssets 의 참조가 매번 변경됨 **위치:** frontend/src/app/(main)/dashboard/page.tsx:269-304 **영향:** WebSocket 배치 업데이트(500ms)마다 3단계 useMemo 연쇄 실행 **권장:** displayAssets 와 filteredDisplayAssets 를 단일 useMemo로 통합 **심각도:** Medium

[PF-L-01] useSearchUsers — queryKey에 excludeIds 배열 직접 포함 (Low)

현상: useSearchUsers(query, excludeIds) 에서 excludeIds 배열이 queryKey에 포함. 배열 참조가 매 렌더마다 새로 생성되면 TanStack Query가 불필요한 리페치 실행 **위치:** frontend/src/hooks/useChat.ts:331 **영향:** 채팅 초대 모달에서 불필요한 사용자 검색 API 호출 **권장:** excludeIds 를 정렬된 문자열(join)로 변환하여 queryKey에 포함 **심각도:** Low

B. API 호출 패턴 — 3건

[API-M-01] 주문 내역 페이지 — 전체 주문 클라이언트 필터링 (Medium)

현상: OrdersPage 에서 useOrders(status) 후 심볼 검색을 클라이언트에서 수행(filteredOrders). 주문이 수천 건인 경우 전체를 프론트엔드로 전달 후 필터링하여 불필요한 네트워크 전송. 서버 페이지네이션 미적용 **위치:**

frontend/src/app/(main)/orders/page.tsx:417-422 **영향:** 대량 주문 사용자에서 초기 로딩 지연 및 메모리 사용 증가 **권장:** 서버 측 symbol 파라미터 필터 + 페이지네이션 추가 **심각도:** Medium

[API-M-02] 체결 내역 — 전체 데이터 클라이언트 필터링 (Medium)

현상: useTradeHistory() 가 전체 체결 내역을 페이지네이션 없이 조회. filteredTrades 에서 side/symbol 필터링을 클라이언트에서 수행. 분석 탭도 동일 전체 데이터 사용 **위치:** frontend/src/hooks/useOrders.ts:182-207 , frontend/src/app/(main)/orders/page.tsx:431-445 **영향:** 체결 내역이 수천 건 이상일 때 응답 크기 및 렌더링 비용 증가 **권장:** 서버 측 limit / offset + side/symbol 파라미터 지원 **심각도:** Medium

[API-L-01] AI 분석 — 뉴스 200개 전체 조회 후 50개만 사용 (Low)

현상: handleAiAnalysis() 에서 뉴스 200개를 조회(limit: 200)한 후 24시간 필터링 + slice(0, 50)으로 최대 50개만 AI에 전달. 150개의 불필요한 데이터 전송 **위치:** frontend/src/app/(main)/dashboard/page.tsx:178-186 **권장:** 서버에서 24시간 이내 뉴스만 필터하여 50개 반환하는 전용 엔드포인트 추가 **심각도:** Low

C. WebSocket 메시지 처리 — 2건

[WS-M-01] AssetDetailPage — 심볼 필터링 콜백에서 startTransition 개별 호출 (Medium)

현상: AssetDetailPage 의 handlePriceUpdate 에서 모든 수신 이벤트마다 startTransition(() => setLivePrice(update)) 호출. 대시보드와 달리 배치 없이 개별 업데이트. 여러 심볼 구독 시 빈번한 transition 생성 **위치:** frontend/src/app/(main)/asset/[symbol]/page.tsx:123-127 **영향:** 고빈도 가격 업데이트 시 불필요한 렌더 스케줄링 **권장:** 대시보드와 동일한 500ms 배치 패턴 적용 또는 ref 기반 throttle **심각도:** Medium

[WS-L-01] 채팅 소켓 — 방 목록 30초 폴링 + WebSocket 이벤트 병행 (Low)

현상: useChatRooms() 가 30초 폴링(refetchInterval: 30000) 사용. 동시에 ChatSocket에서 새 메시지 수신 시 캐시 무효화가 이루어지므로 폴링이 중복. 채팅방이 많으면 불필요한 API 호출 **위치:** frontend/src/hooks/useChat.ts:37 **권장:** WebSocket 메시지 수신 시 캐시 업데이트가 정상적이면 폴링 주기를 60~120초로 완화 **심각도:** Low

D. 번들/코드 분할 — 2건

[BD-M-01] 커뮤니티 페이지 — 950줄 단일 번들 (Medium)

현상: community/page.tsx 가 950줄으로 StrategyCard, TraderCard 등 하위 컴포넌트를 인라인으로 포함. 커뮤니티 페이지 진입 시 전체 코드가 한 번에 로드. 탭별 lazy loading 없음 **위치:**

frontend/src/app/(main)/community/page.tsx **영향:** 초기 번들 크기 증가, 사용하지 않는 탭의 코드도 로드 **권장:** 각 탭 콘텐츠를 dynamic(() => import(...), { ssr: false }) 로 분리 **심각도:** Medium

[BD-L-01] lucide-react 아이콘 — 페이지별 대량 개별 임포트 (Low)

현상: community/page.tsx 에서 16개, orders/page.tsx 에서 11개 아이콘을 개별 임포트. Tree-shaking이 적용되지만, 동일 아이콘이 여러 파일에서 중복 임포트될 경우 번들 코드 중복 **위치:**

frontend/src/app/(main)/community/page.tsx:27-46 ,

frontend/src/app/(main)/orders/page.tsx:26 영향: 전체 번들에서의 영향은 minor하지만 개선 여지 있음 권장: 자주 사용하는 아이콘을 중앙 @/components/icons 모듈로 re-export 심각도: Low

E. DB 쿼리 패턴 — 2건

[DB-H-01] 채팅 getMessages — readReceipts N+1 잠재 문제 (High)

현상: ChatService.getMessages() 에서 메시지 30개를 조회하면서 각 메시지의 readReceipts 를 Prisma select 로 포함. 30개 메시지 x readReceipts 조회로 실질적 N+1 쿼리 가능성. 이후 participantCount 도 별도 쿼리
위치: backend/services/chat/src/chat/chat.service.ts:184-236 영향: 대규모 채팅방에서 메시지 로딩 지연
권장: readReceipts 조인 대신 raw 쿼리로 읽음 여부를 일괄 집계, 또는 _count 만 select 심각도: High

[DB-M-01] 채팅 통계 — 9개 동시 쿼리 실행 (Medium)

현상: ChatService.getStatistics() 에서 Promise.all 로 9개의 Prisma 쿼리를 동시 실행. totalRooms , dmCount , groupCount 등 단순 카운트 쿼리 3개를 단일 raw 쿼리로 통합 가능 위치: backend/services/chat/src/chat/chat.service.ts:527-568 영향: DB 커넥션 풀 점유 증가 권장: 단순 카운트 쿼리를 단일 SELECT COUNT(*) FILTER (WHERE ...) raw 쿼리로 통합 심각도: Medium

F. 이미지/폰트 로딩 — 2건

[IMG-M-01] TipTap 에디터 — Base64 이미지 인라인 저장 (Medium)

현상: RichEditor.tsx 에서 이미지를 Base64로 변환하여 에디터 HTML에 직접 삽입(setImage({ src: base64 })). 5MB 이미지가 Base64로 ~6.7MB 문자열이 되어 DB 게시물 content에 저장. 게시물 목록 조회 시에도 Base64 포함 위치: frontend/src/components/ui/RichEditor.tsx:103-109 영향: DB 저장 용량 증가, 게시물 목록 API 응답 크기 비대, 렌더링 지연 권장: 이미지를 서버 업로드 후 URL 참조로 교체. Base64 인라인 제거 심각도: Medium

[IMG-L-01] CDN 폰트 — preload 미적용 (Low)

현상: layout.tsx 에서 cdn.jsdelivr.net 에 preconnect를 설정했으나, 실제 폰트 파일에 대한 <link rel="preload"> 미적용. 폰트 로드가 CSS 파싱 후에야 시작되어 FOIT 발생 가능 위치: frontend/src/app/layout.tsx:39 권장: 주요 폰트 파일에 <link rel="preload" href="..." as="font" crossorigin> 추가 심각도: Low

G. SSR/Hydration — 1건

[SSR-L-01] auth-prehydrate 스크립트 — sessionStorage 접근 예외 처리 범위 (Low)

현상: layout.tsx:52-67 의 인라인 스크립트에서 sessionStorage 와 localStorage 접근을 각각 try-catch로 감싸지만, Safari 개인 정보 보호 모드 등에서 JSON.parse(null) 이 아닌 storage 자체 접근 차단 시 사일런트 실패. document.documentElement.lang 미설정 시 en 기본값 유지 위치: frontend/src/app/layout.tsx:52-67 권장: storage 접근 실패 시 기본 locale 설정을 보장하는 폴백 추가 심각도: Low

종합 의견

6차 성능 감사에서는 총 18건의 이슈가 발견되었습니다. 5차 감사에서 발견된 주문 엔진 삽입 정렬(OE-H-01), includes() 비효율(OE-M-01), 폴링 중복(FC-M-01/02), WebSocket 재접속(WS-L-01) 등 5건이 정상 수정된 것을 확인하였습니다. AssetList useMemo(FR-H-01)와 리더보드 FLIP(FR-H-02)은 아직 미수정 상태입니다.

High 3건 중 AssetList 정렬 비용과 리더보드 FLIP DOM 조작은 이전 감사에서부터 지속된 이슈이며, 새로 발견된 채팅 readReceipts N+1 문제(DB-H-01)는 대규모 채팅방에서 메시지 로딩 성능에 직접적 영향을 미칩니다.

TipTap 에디터의 Base64 이미지 인라인 저장(IMG-M-01)은 DB 용량과 API 응답 크기 모두에 영향을 미치므로 이미지 업로드 방식으로의 전환을 권장합니다. 커뮤니티 페이지의 950줄 단일 컴포넌트(PF-M-01, BD-M-01)는 코드 분할과 렌더링 최적화를 동시에 해결할 수 있는 항목입니다.

본 보고서는 자동 생성된 감사 결과이며, 수정 사항은 포함되지 않습니다.